The code surface is now genuinely defended. Every remaining bypass is an unsigned JSON file that a gate trusts.

Third independent red-team pass. All four round-2 bypasses are closed and verified closed. What remains is a single, sharply-defined failure class — and it has one fix.

FLOW_BLOCKED

Eleven items fixed, including every round-2 bypass. But a fully forged smoke acceptance — declaring *4,321 look-ahead violations* and pointing at a nonexistent run — still authorizes a full production run.

FIXED THIS ROUND	R2 BYPASSES CLOSED	STILL OPEN	NEW FINDINGS	SEALED FILES
11	4 / 4	3	3	37 → 53

EXECUTIVE VERDICT

A real step change, and one failure class left

This round is materially different from the last one. Where round 2's governance fixes were written to the text of my findings, **round 3 fixed the underlying properties**. The resolver now performs an actual AST transitive closure over three dependency graphs — runtime, contract authority, and governance — and raises `UnresolvedDependencyError` on any repo-local import it cannot resolve. The sealed set went from 37 hand-listed files to **53 computed ones**. I re-ran every round-2 bypass and all four are now blocked.

Several fixes went beyond what I asked for. `timestamp_engine.py` no longer asserts the timestamp contract — it opens catalog parquet, measures `ts_init - ts_event`, and raises `CatalogTimestampSemanticError` on violation. The warmup guard now builds a proper year *range* and checks it against both prohibited years and the DEV lock, blocking my round-2 leak with a precise error. The two `except ImportError: pass` fail-opens on the OOS lock now re-raise. And the smoke acceptance a real

validator produced is genuinely good evidence: `2,002 candidates, 0 duplicates, 0 future-source violations, exact feature SHA` — independently corroborating that F3 is fixed.

> **The causal surface is now in good shape.** F3 (the look-ahead violation) remains fixed and survived this round's edits; the lint covers 46/46 Python files at 100%; the closure covers 53 files; every code-tampering bypass I have attempted across three rounds is now blocked. If the remaining items were closed, I would sign this off.

What remains is one failure class, stated precisely: three JSON files — `smoke_acceptance.json`, `status.json`, `contract_status.json` — carry authorization decisions, are consumed by gates that trust their contents, and are *not covered by the seal and not bound to any producer*. Anyone who can write to the study directory can author all three by hand. The fix is one pattern applied three times: seal them, and bind each to the artifact it summarizes.

Note the asymmetry that makes this so easy to miss: the round-3 work correctly identified that *code* must be inside the closure, and built real machinery to guarantee it. But the closure is a *Python import graph*, so authorization data that no module imports is structurally invisible to it — no matter how good the AST walk is.

ROUND-2 FINDINGS, RE-TESTED

Every bypass re-attempted against the new closure

ID	FINDING	EVIDENCE	STATUS
N1	Smoke gate read the wrong seal key	Now `seal_data["composite_seal_hash"]` with an explicit key-presence check, plus study-name match and a `deterministic_validation_verified` requirement	FIXED
N2	`generate_oos_unlock.py` unsealed	In the governance closure. Tampered → `PREEXEC_AUDIT_STALE`	FIXED
N3	`study_spec.py` unsealed	In the contract-authority closure. Tampered → `PREEXEC_AUDIT_STALE`	FIXED
N4	Closure test was a filename checklist	Real `compute_ast_closure()`; `UnresolvedDependencyError` on unresolvable repo-local imports; 25 tests pass	FIXED
N5	Strategy path built from a dotted class name	`resolve_dynamic_strategy_file()` splits correctly and raises `UnresolvedStrategyError`	FIXED
N6	Warmup entered locked DEV years	Verified blocked: `UNAUTHORIZED_WARMUP_DOMAIN: Warmup window [2024-12-27 to 2025-01-01] enters locked DEV year 2024`	FIXED
F2	Audit provenance self-issued	Verdict now *parsed* from the report rather than passed in — a real improvement. But the parser is optional and the parse itself fails open (R3-2, R3-3)	PARTIAL

ID	FINDING	EVIDENCE	STATUS
F6	Warmup crosses unauthorized years	Prohibited and DEV now checked; **authorized-set membership still is not**. Real study full-stage still warms from unauthorized 2020 (R3-4)	PARTIAL
F7	SPEC fidelity hardcoded to one study	Per-study `study_clauses.yaml` introduced and sealed — good. But SPEC.md is still never opened, and an absent clause file falls back to three trivial non-empty checks (R3-6)	PARTIAL
F11	OOS unlock fail-opens	Both `except ImportError: pass` now raise `OOS_AUTHORIZATION_DEPENDENCY_MISSING`. The hand-typed `2024_not_read…` boolean remains	PARTIAL
F12	Timestamp engine asserted, didn't measure	`measure_catalog_bar_semantics()` reads catalog parquet and raises `CatalogTimestampSemanticError` on delta violation	FIXED
F13	Auditor has Bash+Write; hook unwired	Unchanged: `tools: [Read, Grep, Glob, Bash, Write]`; `settings.local.json` still has no `hooks` key	OPEN
F14	Study-specific literals in framework	`feature_universe_count = 60` removed; tracker list now derived. `meta_cols` in `output_manager.py:118` still hardcoded	PARTIAL
F17	Stale runs indistinguishable	No lineage marker on `runs/`; the acceptance now records `run_dir` + parquet SHA, which is most of the fix — but nothing verifies them (R3-1)	PARTIAL
F3 / F4	Causal fixes from round 2	Both intact after this round's edits. Skip-on-gap present at `collector:393–397`; `obs_ts = T` at `:500`; lint 46/46 at 100%	HOLDS

BYPASS ATTEMPTS

Executed against the live repository, then reverted

CLOSED — THE THREE ROUND-2 CODE BYPASSES

```
$ # append a comment to each of the three previously-unsealed modules
re-tampered the 3 round-2 bypass targets

$ python scripts/preexec_audit_seal.py --study studies/<s> --verify-only
PreexecAuditStaleError: PREEXEC_AUDIT_STALE: File 'research/schemas/study_spec.py' was
  modified after audit seal! (Sealed hash: fc4d27682990..., Current hash: 6b1894eb97b0...)
```

CLOSED — THE ROUND-2 WARMUP LEAK

```
N6 DEV-adjacent warmup -> BLOCKED: UnauthorizedExecutionDomainError
    UNAUTHORIZED_WARMUP_DOMAIN: Warmup window [2024-12-27 to 2025-01-01] enters
    locked DEV year 2024 without authorized OOS unlock token.
```

STILL OPEN — FORGED SMOKE ACCEPTANCE AUTHORIZES A FULL RUN

```
$ # write smoke_acceptance.json by hand. No smoke was run. All fields are
$ # derivable from public repo state; the file is not in the seal.
```

```
RESULT: stage=FULL ACCEPTED a fully forged smoke acceptance.
        run_dir points nowhere, parquet sha is zeros,
        future_source_violations_count = 4321  (self-declared FAILING smoke)
        -> reached engine build anyway.
```

STILL OPEN — HAND-WRITTEN AUDIT STATUS BYPASSES THE NEW PARSER ENTIRELY

```
$ # write status.json and contract_status.json by hand, declare CLEAR,
$ # use legacy provenance version 1. The parser is never invoked.
RESULT: seal GENERATED from hand-written status.json (no parser, no auditor).
        seal_status = LOCKED | composite = 24d608f344472ddc...
        provenance version accepted: 1
```

STILL OPEN — THE REPORT PARSER READS A BLOCKING AUDIT AS CLEAR

```
A. prose CRITICAL findings, no numeric summary, no verdict token
    -> RAISED AuditArtifactParseError (fail-closed)
B. prose CRITICAL findings + the word CLEAR in benign prose
    -> verdict=CLEAR  critical=0  warning=0   <== FAIL-OPEN
C. low count stated first, true tally later
    -> verdict=CLEAR  critical=0  warning=7
D. one-word report ("CLEAR\n")
    -> verdict=CLEAR  critical=0  warning=0
E. honest blocked report
    -> verdict=BLOCKED critical=2
```

FINDINGS

Three criticals, three warnings

CRITICAL R3-1

The smoke acceptance is unsealed, and none of its substantive claims are re-verified

`backtests/nt_runtime/modes/collect.py:41-75` · `scripts/resolve_execution_manifest.py:182-210`

MECHANISM

`resolve_study_files()` seals five mandatory study files plus `study_clauses.yaml` and tests — `artifacts/smoke_acceptance.json` is not among them. The gate then checks four fields: `status`, `study_name`, `sealed_composite_sha256`, and `deterministic_validation_verified`. Every one is derivable from public repo state, and the file's own substantive fields — `run_dir`, `candidates_parquet_sha256`, `future_source_violations_count`, `duplicate_candidates_count` — are recorded but never read by the gate.

FAILURE

Demonstrated end-to-end. A hand-written acceptance declaring **4,321 future-source violations**, a zeroed parquet hash and `run_dir: /nonexistent/never/ran` passed the gate and reached engine construction. The validator that produces the honest version (`scripts/validate_smoke.py`) is sealed and appears sound — but nothing forces its output to be the file the gate reads.

WHY MISSED

The closure is an import graph. `smoke_acceptance.json` is data no module imports, so no amount of AST work reaches it. Its correct home is `resolve_study_files()`, which is hand-listed.

FIX

Two parts, both small. (1) Have the gate *enforce* the fields the validator already writes: `future_source_violations_count == 0`, `duplicate_candidates_count == 0`, `exact_timestamp_equality_verified is True`, and re-hash the parquet at `run_dir` against `candidates_parquet_sha256`. (2) Because the acceptance is written *after* sealing, it cannot go in the seal — instead have `validate_smoke.py` record `validator_file_sha256` (it already does) and have the gate verify that value against the sealed validator's actual hash, so an acceptance can only claim to come from the sealed validator.

REGRESSION TEST

`test_full_stage_rejects_forged_smoke_acceptance` — an acceptance with non-zero violation counts, and one whose `candidates_parquet_sha256` does not match the file at `run_dir`, must both raise.

CRITICAL R3-2

The new audit parser is optional — a hand-written status.json still produces a valid seal

`scripts/preexec_audit_seal.py:84-87` · `scripts/run_preexec_audits.py:127-150`

MECHANISM

Moving the verdict from a function parameter to a parsed value closes the *legitimate* path's abuse, but nothing closes the illegitimate one. The seal validates `status.json`'s *shape* — provenance version, report existence, report SHA, verdict, counts, composite match — all of which a hand-authored file satisfies. `derived_by_parser` is a free-text string the seal never checks. Worse, the version gate was widened to `not in (1, 2)`, so **legacy version-1 records — the ones with orchestrator-declared verdicts — remain acceptable**.

FAILURE

Demonstrated: wrote both status files by hand with `"auditor": "definitely_a_real_independent_auditor"` and `audit_provenance_version: 1`, and `generate_preexec_audit_seal()` returned `seal_status: LOCKED`.

FIX

Require `audit_provenance_version == 2` (drop 1 — the legacy records it admits are exactly the untrustworthy ones). Then make the seal re-derive rather than trust: call `parse_causal_audit_report()` on the sealed report itself and require the parsed verdict and counts to equal what `status.json` claims. That makes the JSON a cache of the parse instead of an independent assertion, and costs about six lines.

REGRESSION TEST

`test_seal_rejects_handwritten_status` — a status whose verdict disagrees with a re-parse of its own report must raise.

CRITICAL R3-3

The report parser records a blocking audit as CLEAR when findings are prose rather than counts

`scripts/run_preexec_audits.py:46-70, :82-101`

MECHANISM

Three compounding weaknesses. The verdict falls back to `re.search(r"\b(CLEAR|BLOCKED)\b", text)` — the first occurrence anywhere, including incidental prose. The critical count requires `Critical` followed by a **digit**; a report whose findings are headed `### CRITICAL: …` matches nothing, and the count silently defaults to `0` whenever the verdict token read CLEAR. And `re.search` takes the first match, so an early "Critical: 0" beats a later true tally.

FAILURE

Measured. A report containing two genuine `### CRITICAL:` findings and one benign sentence — "the execution surface is CLEAR of scratch dependencies" — parses as **verdict=CLEAR, critical=0**. A report with "Critical: 0" early and `| CRITICAL | 4 |` later also parses as CLEAR. A file containing only the word `CLEAR` issues a CLEAR status. Note that all three of my own audit reports would parse as CLEAR under this rule.

WHY MISSED

The parser was tested against well-formed reports. Its failure mode requires a report that is *realistic* rather than schematic.

FIX

Stop parsing prose. Require a machine-readable block the auditor must emit — a fenced `audit-summary` JSON with `verdict`, `critical`, `warning`, `note` — and raise `AuditArtifactParseError` when it is absent rather than inferring from free text. As a defence in depth, also count `^#{1,6}\s*CRITICAL` headings and raise on any mismatch with the declared count.

REGRESSION TEST

`test_parser_blocks_on_prose_critical_findings` — the four cases above, each asserted to raise or return BLOCKED.

WARNING R3-4

Warmup still is not checked against the authorized set

`backtests/nt_runtime/data_plan.py:142–167`

MECHANISM	The active-window guard applies three tests: prohibited, authorized-set membership, and the DEV lock. The warmup guard now applies two of them — prohibited and DEV — but still omits authorized-set membership.
FAILURE	Verified: the real study at `--stage full` resolves warmup to **2020-12-27**. 2020 is neither prohibited nor DEV, but it is not in `{2021, 2022, 2023, 2024}` either, so five days of unauthorized data enter the replay. The consequence here is small; the gap is that "not explicitly forbidden" is being treated as "authorized" for warmup only.
FIX	Fold `warmup_years` into `run_years` before the three checks so all of them apply uniformly, or add the authorized-set test to the warmup block. If pre-partition warmup is intentional, make it an explicit `chronology.warmup` allowance rather than an unchecked default.
REGRESSION TEST	`test_warmup_year_must_be_authorized`.

WARNING R3-5

coverage_pct is a hardcoded literal, so the coverage metric cannot fail

`scripts/resolve_execution_manifest.py:320-328`

MECHANISM

`files_count = len(file_hashes)`, then `files_expected = files_count`, `files_resolved = files_count`, `"coverage_pct": 100.0`, `"unresolved_dependencies": []` — all written unconditionally after the point where a failure would already have raised. The reported figure is "everything I found divided by everything I found."

FAILURE

Not exploitable on its own — the genuine control is the `UnresolvedDependencyError` raise at line 280, which does work. The risk is that `Coverage: 100.0%` on the summary card and in `execution_manifest.json` reads as an independently measured guarantee when it is a constant. That is the same pattern that made round 2's closure test misleading, in a lower-stakes place.

FIX

Either compute it honestly — `resolved / (resolved + len(all_unresolved))` before the raise — or delete the three fields and let the exception be the whole contract. Deleting is cleaner.

REGRESSION TEST

`test_coverage_pct_reflects_unresolved`, or removal of the assertion alongside the fields.

WARNING R3-6

SPEC fidelity is opt-in, and still never opens SPEC.md

`scripts/check_spec_fidelity.py:83-108`

MECHANISM	Per-study `study_clauses.yaml` is the right design and removes the hardcoded years from framework code — a genuine improvement, and the file is sealed. But when it is absent, `load_study_clauses()` falls back to three checks that assert `chronology.train`, `chronology.prohibited` and `instrument.symbol` are non-empty. A study that ships no clause file therefore passes SPEC fidelity at 100% coverage for having non-empty fields. Despite the module docstring's "from study_clauses.yaml (or SPEC.md)", `SPEC.md` is still never read.
FIX	Make `study_clauses.yaml` mandatory — raise when absent rather than degrading — and have `create_study.py` scaffold it. Since the resolver already seals it when present, requiring it also closes the gap where dropping the file weakens the gate without changing the composite.
REGRESSION TEST	`test_spec_fidelity_requires_clause_file`.

RECOMMENDED ORDER

Three criticals, all small, all the same shape

1. **R3-1** — the only finding that authorizes a full production run on bad evidence. Enforce the fields the validator already writes, and bind the acceptance to the sealed validator's hash.
2. **R3-3** — require a structured `audit-summary` block and stop inferring verdicts from prose. Without this, an auditor that finds real defects can be recorded as CLEAR.
3. **R3-2** — require provenance version 2, and have the seal re-parse the report rather than trust the JSON.
4. **R3-4, R3-5, R3-6** — each a few lines.
5. **F13** — still unaddressed across three rounds: `lookahead-auditor` holds `Bash` and `Write`, and the `results-triager` guard hook sits on disk unregistered. Given that R3-2 shows audit status is hand-writable, an auditor with shell access can issue its own PASS.

A closing observation on where the boundary now sits. Round 3 correctly concluded that the seal must cover a computed closure rather than a list, and built the machinery

to do it. But a Python import graph can only ever reach code. The three files that still carry authorization — an acceptance, a causal status, a contract status — are read as data, produced after the seal, and referenced by no import. **They need a second mechanism: not "is this file in the closure?" but "was this file produced by the sealed tool that claims to have produced it?"** Each of the three already records enough to answer that (`validator_file_sha256`, `audit_report_sha256`, `derived_by_parser`); the gates simply do not check.

Red team round 3, 2026-08-15 · study `Gemini_clean_maturity_flip_rolling_5m_productivity` · seal composite `24d608f344472ddc…` · 53 sealed files · lint 46/46 · 25/25 tests pass. All bypasses executed against the live working tree and reverted; `PREEXEC_AUDIT_SEAL_VALID`, `PREFLIGHT CLEAR` and clean file state reconfirmed after restore. The genuine `smoke_acceptance.json` was backed up before testing and restored intact (2,002 candidates, 0 violations).